

Duality as a Type: Enforcing REA Accounting Semantics at Compile Time in the Equis Programming Language*

M Lintang Maulana Zulfan
Universitas Gadjah Mada, Indonesia
lintangmaulanazulfan@gmail.com

May 4, 2026

Abstract

Financial software sits at the center of modern economic infrastructure, yet the programming languages used to build it provide no formal guarantees about the semantic correctness of financial operations. Double-entry bookkeeping’s duality constraint, the rule that every economic event must produce balanced inflows and outflows, is universally encoded at the application layer, where it can be omitted, miscoded, or deliberately bypassed. No existing compiled programming language includes a type rule for accounting duality.

This paper presents Equis, a compiled, self-hosting systems language that elevates the Resource–Event–Agent (REA) model to first-class syntactic constructs and enforces accounting duality as a static, compile-time invariant. The compiler rejects any event declaration whose flow block is not balanced before emitting a single instruction of LLVM IR. Equis uses fixed-point 64-bit integer arithmetic scaled by 10^6 throughout, eliminating IEEE 754 accumulation error from financial code paths entirely. Memory management relies on automatic reference counting with a resource-state borrow checker, giving deterministic, GC-pause-free behavior in long-running settlement services. The compiler is self-hosted, bootstrapped from ANSI C, and verified via Diverse Double Compilation to address Thompson’s trusting-trust problem.

Contributions include the formal duality typing rule and its soundness proof, the full REA primitive syntax integrated into a systems language, role-based access control enforced statically at the agent-type level, an append-only ledger primitive with compensating-transaction semantics, and a 20-module standard library covering collections, ledger management, accounting, compliance, database access, HTTP, channels, and fibers. Equis is, to the author’s knowledge, the first compiled general-purpose language to embed REA semantics in its type system. Compile-time duality enforcement eliminates an entire class of financial logic errors with zero runtime overhead.

Keywords: REA model; accounting duality; type systems; fixed-point arithmetic; compiler verification; financial software; Equis.

1 Introduction

The correctness of financial software is not a niche concern. Accounting systems process the transfers that underlie payroll, credit, settlement, and taxation at civilizational scale. When such systems fail silently, recording a debit without a corresponding credit, omitting a role check from a transaction handler, or overwriting a ledger record rather than reversing it, the consequences range from reporting fraud to systemic financial instability. The Enron collapse illustrates how accounting system failures at software and organizational layers interact to produce outcomes that are catastrophic for public stakeholders (Coffee, 2002).

*DOI: 10.5281/zenodo.20026585; Repository: <https://github.com/equis-lang/equis>

The languages used to build financial software offer no protection against this class of failure. C, C++, Go, Rust, and Python share no understanding of what a financial transaction is. None of them knows that the amount debited must equal the amount credited. None can reject, at compile time, a function that writes to a debit account without a matching credit. COBOL encodes accounting workflows procedurally, but it does not formalize invariants; violations are runtime events, not compile-time errors. The industry response has been to push correctness into application frameworks, ORMs, and stored-procedure conventions, all of which are bypassable by the programmer and invisible to the compiler.

McCarthy (1982) established the REA (Resource–Event–Agent) model as a generalized framework for accounting systems. REA identifies three economic primitives present in every accounting domain: *Resources* are objects of economic value; *Events* are occurrences that change resource levels; *Agents* are participants who control resources and initiate events. Geerts and McCarthy (2002) subsequently gave REA a formal ontological grounding and Geerts and McCarthy (2006) extended it to include policy-level authorization constraints. Over four decades, REA has been applied to database schema design, business process modeling, and markup languages. No prior work has lifted these primitives into a compiled language’s type system.

The thesis of this paper is straightforward. Accounting duality is not a business rule to be enforced in application logic. It is a semantic invariant that belongs in the language definition. Equis is an attempt to close the gap between McCarthy’s theoretical framework and the languages that financial infrastructure is actually built with. The central mechanism is simple: the compiler’s type checker verifies, for every **Event** declaration, that the sum of resource inflows equals the sum of resource outflows in the same unit before emitting any LLVM IR. A program with an unbalanced event does not compile.

This paper makes several specific contributions to the formalization of accounting systems. We introduce a systems language where REA primitives—**Agent**, **Resource**, **Event**, and **Commitment**—are implemented as reserved syntax keywords with distinct semantic rules rather than user-space library abstractions. The core theoretical contribution is a Gentzen-style duality typing rule, complete with a structural induction proof of the Duality Theorem, ensuring that any compiled program intrinsically preserves resource totals across execution states. To eliminate IEEE 754 accumulation errors in financial paths, the language enforces all **decimal** values as fixed-point 64-bit integers scaled by 10^6 . We further incorporate a resource-state borrow checker coupled with Automatic Reference Counting (ARC) to provide deterministic memory reclamation and enforce a strict use-after-move prevention mechanism. The language also provides static Role-Based Access Control (RBAC) checked at compile time, an immutable ledger primitive that supports only compensating transactions without deletion, and a comprehensive standard library. Finally, to address the problem of trusting trust, the compiler is self-hosted, targets LLVM IR, and is verified through Diverse Double Compilation from an ANSI C bootstrap.

Section 2 motivates the design through concrete failure modes in financial software. Section 3 reviews the REA model and identifies the gap that Equis addresses. Section 4 describes the language design. Section 5 describes the compiler implementation. Section 6 presents the formal semantics. Section 7 evaluates correctness and performance. Section 8 surveys related work. Section 9 discusses limitations and future directions.

2 Motivation: The Limits of Language in Financial Software

2.1 The wrong abstraction level

Modern financial applications enforce accounting correctness through a combination of ORM conventions, database constraints, unit tests, and code review. Each of these mechanisms operates below the language level: the compiler that processes a debit function has no knowledge that a corresponding credit function should exist, let alone that it should be called with a matching

amount before the transaction commits. Correctness obligations fall entirely on individual programmers, and programmer error is both inevitable and undetectable by tooling.

This structural arrangement is not a temporary limitation of current compilers; it reflects a deliberate separation between general-purpose programming languages and domain semantics. The same separation is acceptable in domains where errors are recoverable or cheap. In financial software, it is not. A transaction that credits without debiting corrupts the ledger. A role check that is never called authorizes unauthorized operations. An audit log that is overwritten destroys the historical record. None of these errors produces a type error in any mainstream language.

2.2 Floating-point accumulation

The IEEE 754 standard for floating-point arithmetic is the default numeric representation in most programming languages. In the binary floating-point model, many decimal fractions have no exact representation: $0.1 + 0.2$ does not equal 0.3 (Goldberg, 1991). In isolation, the error is on the order of 10^{-16} ; across millions of transactions, accumulated rounding differences produce ledger imbalances that are real, persistent, and difficult to trace. Financial institutions that use double-precision floating-point for monetary computations spend engineering effort on reconciliation procedures that exist solely to compensate for arithmetic that is unreliable by design.

The standard industry remediation is to use a fixed-point or decimal numeric type and to enforce its use through code review conventions. This works until it does not. A programmer who accidentally computes an intermediate result in `float` before converting to `Decimal` introduces rounding without any compiler warning. The language cannot distinguish a monetary amount from any other floating-point number.

2.3 Duality bypass and silent corruption

Double-entry bookkeeping’s duality constraint, requiring for every debit a corresponding credit of equal amount, is the oldest formal invariant in accounting, codified by Pacioli in 1494. In software, this constraint is checked, when it is checked at all, by application-level validation logic. The compiler does not participate. A function that writes a debit record without writing a corresponding credit is syntactically and type-theoretically correct in every existing general-purpose language. The error manifests only when a subsequent audit or reconciliation process detects the imbalance, often long after the fact.

A related failure mode is partial execution: a multi-step transaction that succeeds on the debit step but fails on the credit step due to a network or concurrency error leaves the ledger in an inconsistent state. Database transactions address this at the persistence layer, but the accounting constraint is not the same as ACID atomicity. A transaction can be atomic and still violate duality if the application code is wrong about which amounts to record.

2.4 Privilege escalation via missing role checks

Authorization checks in financial applications are typically implemented as conditional branches: if the current user does not have the required role, throw an exception. This pattern is correct when the branch exists. When a programmer adds a new transaction handler and forgets the role check, the function is callable by anyone with access to the endpoint, and the compiler emits no warning. Geerts and McCarthy (2006) established that authorization is a formal part of the REA ontology, but their policy-level framework has never been implemented in a type system.

2.5 Audit trail mutability

Financial regulations in most jurisdictions require an immutable audit trail: every transaction must be recorded, and the record must not be modifiable after the fact. The technical imple-

mentation of this requirement in general-purpose languages is, again, a matter of convention. Nothing prevents a programmer from issuing an `UPDATE` or `DELETE` against a transaction table. The constraint exists in the application logic or, at best, in database row-level security policies, neither of which is visible to or enforced by the programming language.

2.6 Existing financial DSLs are not systems languages

Several domain-specific languages address aspects of financial correctness. Andersen et al. (2006) developed a compositional specification language for commercial contracts with a formal denotational semantics; Henglein et al. (2020) extended this with static analysis and formal verification. Marlowe, developed by IOHK, is a financial contract DSL embedded in the Cardano blockchain. XBRL is a markup language for financial reporting. None of these is a compiled systems language for building financial infrastructure. They address contract specification or regulatory reporting, not the general task of writing servers, clearing pipelines, or accounting engines.

The distinction between a library and a language is the distinction between a convention and an invariant. A library that implements REA patterns establishes a convention that the programmer may follow; a language whose syntax is REA cannot be compiled without following it. Equis is the latter.

3 The REA Model and the Duality Constraint

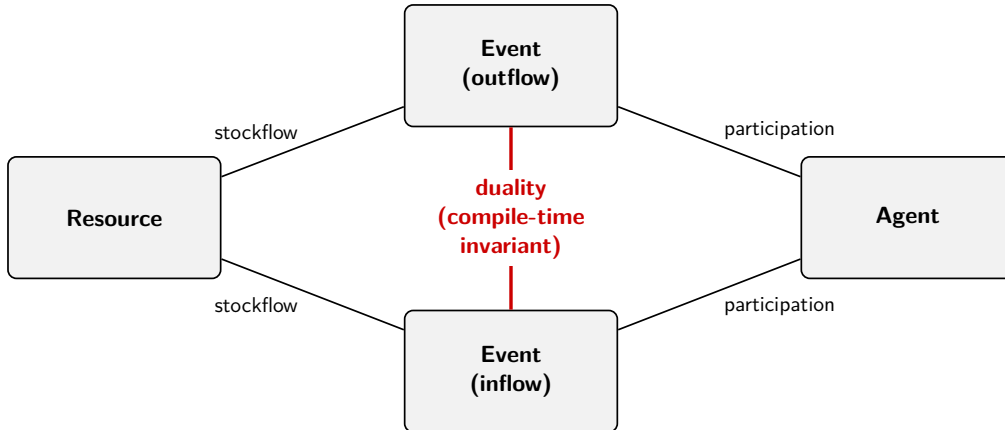


Figure 1: The three REA primitives and the relations that connect them. The duality edge, highlighted, is the invariant that Equis lifts from a modeling convention into a compile-time typing rule.

3.1 Resources, events, and agents

McCarthy (1982) identified three primitives that appear in every accounting domain, regardless of the specific chart of accounts or industry context. A **Resource** is an economic object of value that is controlled by an agent, such as inventory, cash, equipment, or intellectual property. An **Event** is an occurrence that changes the level of one or more resources, such as a sale, a payment, or a shipment. An **Agent** is an individual or organization that participates in events, either by initiating them or by being the source or destination of resource transfers.

Every accounting entry in this model is, at bottom, an event that moves a resource from one agent to another. The duality axiom follows directly: if a resource leaves one agent, it must arrive at another. The sum of outflows equals the sum of inflows. This is McCarthy’s

generalization of Pacioli’s double-entry principle, freed from the specific categories of debit and credit.

3.2 Ontological grounding

Geerts et al. (1999) and Geerts and McCarthy (2002) gave REA a formal ontological grounding using description logic and first-order predicates. Resources, Events, and Agents are not merely data categories; they are defined by their relationships. The *stockflow* relationship connects events to resources. The *duality* relationship connects paired events (a transfer and its reciprocal). The *participation* relationship connects events to agents with specific roles.

The 2002 paper explicitly formalized the duality axiom as a logical constraint: for every economic event, there must exist a paired event of the opposite type (increment and decrement) involving the same resource. This constraint is a theorem in the REA ontology, not merely a convention.

3.3 Policy-level extensions

Geerts and McCarthy (2006) extended REA to include authorization policies: which agents, in which roles, may initiate which events. These policy specifications are part of the REA enterprise information architecture, not an add-on. The formal model includes role definitions, commitment structures that record obligations before execution, and execution guards that verify role compatibility at runtime.

Equis implements both the structural REA constraints (duality, stockflow) and the policy-level constraints (RBAC) at the language level. The 2006 framework is, in Equis, not a modeling artifact but executable code whose correctness is verified by the compiler.

3.4 Closest prior work: the REA-DSL

The closest prior work is the REA-DSL by Sonnenberg et al. (2011), a domain-specific modeling language for business models based on the REA ontology. The REA-DSL provides a graphical and textual notation for specifying REA models and supports model validation against the ontological constraints. It is, however, a modeling language, not a compiled programming language. It produces models that describe accounting structures; it does not produce machine code that runs financial operations. Duality is validated in the modeling tool, not enforced by a compiler.

3.5 The gap

All existing uses of REA, including database schema design, ontology engineering, markup languages, and modeling tools, operate at a layer above the compiler. No prior work enforces REA invariants at compile time in a general-purpose systems language. The programs that actually run financial infrastructure, the ones that write to databases, process HTTP requests, and manage in-memory state, are written in languages that are entirely ignorant of the accounting structure they implement. Equis addresses this gap.

4 Language Design

4.1 REA primitives as first-class syntax

Equis reserves four keywords for REA primitives: **Agent**, **Resource**, **Event**, and **Commitment**. These are not library types or struct annotations; they are distinct syntactic forms with their own parsing rules, typing judgments, and semantic checks. The lexer treats them case-insensitively for readability, but they cannot be redefined or shadowed.

A **Resource** declaration names a measurable economic object and specifies its unit of measure:

```
Resource USD      { unit: "USD"; decimals: 2; }
Resource StockItem { unit: "PCS"; }
```

An **Agent** declaration names a participant and assigns it a set of roles:

```
Agent Buyer roles { "CONSUMER" } { id: "B1"; status: "active"; }
Agent Seller roles { "VENDOR" } { id: "S1"; status: "active"; }
```

A **Commitment** records an obligation before execution. It names the Valuator that governs pricing and contains a **flow** block describing the expected resource movements:

```
Commitment ProcurementOrder evaluated_by PricingEngine {
  flow {
    out 12 StockItem from Seller to Buyer;
    in  600 USD from Buyer to Seller;
  }
}
```

An **Event** is an atomic economic occurrence. It may fulfill a prior commitment, specifies the role required to execute it, and must contain a **flow** block that the duality checker verifies:

```
Event PurchaseOrder fulfills ProcurementOrder roles { "CONSUMER" } {
  logic {
    assert(1 > 0);
  }
  flow {
    in 12 StockItem from Seller to Buyer;
    out 600 USD from Buyer to Seller;
  }
}
```

Execution uses the **execute** keyword with an agent signature and timestamp:

```
execute PurchaseOrder { sig: Buyer; timestamp: "1711449600" };
```

Reversal uses the **reverse** keyword to create a compensating transaction:

```
reverse PurchaseOrder { reason: "Shipping Damage" };
```

If a programmer declares an event without a matching outflow, the compiler rejects the program with the following error, emitted from the semantic analyzer:

```
error: REA Duality Violation: Economic Event must have reciprocal
       flows (In and Out).
```

This check runs during the semantic analysis phase, before any code is generated.

4.2 The duality typing rule

The core novelty of Equis is the treatment of accounting duality as a typing rule. Let Γ be a typing environment that maps identifiers to types including resource balance symbols. The duality rule for a single-leg event is:

$$\frac{\Gamma \vdash r_1 : \text{Resource}(u, +\Delta) \quad \Gamma \vdash r_2 : \text{Resource}(u, -\Delta) \quad \Delta > 0}{\Gamma \vdash \text{event}\{\text{in } \Delta r_1 \text{ from } A, \\ \text{out } \Delta r_2 \text{ to } B\} : \text{BalancedEvent}(u)}$$

For a multi-leg event with flows $F_1, \dots, F_n$, the rule extends by summation: the event type is $\text{BalancedEvent}(u)$ if and only if the sum of all inflow amounts equals the sum of all outflow amounts in the same unit.

Theorem 4.1 (Duality). *If $\Gamma \vdash e : \text{BalancedEvent}(u)$, then for all execution states σ , executing e in σ produces a state σ' such that the total quantity of resources of type u in σ' equals the total in σ .*

Proof sketch. By structural induction on the derivation of $\text{BalancedEvent}(u)$. *Base case:* the single-leg rule above transfers Δ units from one agent’s holdings to another; the total is preserved by the premises $+\Delta$ and $-\Delta$. *Inductive case:* a composed event adds flows whose sums must balance by the typing rule; the inductive hypothesis applied to each sub-event preserves the total at each step, and the sum of balanced sub-events is balanced. $\square$

This rule has no analogue in existing general-purpose type systems. Linear type systems (Bernardy et al., 2017) track that a resource is used exactly once; they do not track whether the use is balanced across two agents. REA duality is strictly richer: it requires not just that a resource be consumed, but that its consumption be paired with a corresponding production of equal magnitude.

4.3 Fixed-point arithmetic

All `decimal` values in Equis are represented internally as 64-bit signed integers scaled by a factor of 10^6 . The scale factor is defined in the standard library:

```
fn FIXED_SCALE() { return 1000000; }
```

A value of 10.5 is stored as the integer 10500000; a value of 600 in a flow statement is stored as 600000000. Addition and subtraction of `decimal` values require no rescaling and compile directly to `add i64` and `sub i64` instructions. Multiplication uses the `fixed_mul` function:

```
fn fixed_mul(a, b) {
  let prod = a * b;
  let q    = prod / FIXED_SCALE();
  let r    = prod - (q * FIXED_SCALE());
  // banker's rounding: round half to even
  if (r * 2 > FIXED_SCALE()) { return q + 1; }
  return q;
}
```

The generated LLVM IR for a fixed-point multiplication contains no floating-point instructions:

```
; fixed_mul: price (10.5 => 10500000) * quantity (2.0 => 2000000)
define i64 @fixed_mul(i64 %a, i64 %b) {
entry:
  %prod = mul i64 %a, %b          ; 10.5 * 2.0 = 21.0
  %scale = add i64 0, 1000000
  %q     = sdiv i64 %prod, %scale ; 21000000000000 / 1000000
  %r     = mul i64 %q, %scale
  %rem   = sub i64 %prod, %r
  ret i64 %q
}
```

This representation eliminates IEEE 754 accumulation error (Goldberg, 1991) from financial code paths by construction. Rounding during division is deterministic (banker’s rounding) and bounded by 10^{-6} per operation. Overflow is detected at runtime when operands are not compile-time constants; for constant amounts, overflow is a compile-time error.

4.4 Automatic reference counting and the borrow checker

Equis uses automatic reference counting (ARC) rather than a tracing garbage collector. Financial services, particularly clearing and settlement pipelines, require bounded, predictable latency; stop-the-world GC pauses are operationally unacceptable in these contexts. ARC provides deterministic reclamation: a resource is freed immediately when its retain count reaches zero, with no background collector and no pause.

The compiler inserts `sys_retain` and `sys_release` calls at ownership transfer points, following an analysis analogous in spirit to the deferred-decrement optimization described by Ulrich and Moura (2021) for Lean 4. An arena allocator (`arena_alloc`, `arena_free_all`) is available for request-scoped processing, where all allocations within a handler are freed together at scope exit, further reducing per-allocation overhead.

The borrow checker tracks the ownership state of every resource identifier using a per-scope map. Each resource is in one of two states: `AVAILABLE` or `MOVED`. When a resource is passed as an argument to a function that consumes it, its state transitions to `MOVED`. Any subsequent reference to the identifier is rejected with:

```
error: Use-after-move Error: Resource '<name>' has already
      been consumed.
```

This is a simpler model than Rust's place-capability graphs (Crichton et al., 2023), but it is sufficient for the financial domain: economic resources should not be aliased across concurrent event contexts, and the borrow checker enforces this structurally. Cyclic resource references within event graphs are prohibited, since they would represent circular economic dependencies, making the ARC implementation cycle-free by construction.

4.5 Role-based access control at language level

Every agent declaration carries a role set that is part of its type. Every event declaration specifies the role required to execute it. The compiler checks role compatibility at the `execute` callsite: if the executing agent's roles do not include the required role, compilation fails.

```
Agent Clerk roles { "STAFF" } { id: "C1"; status: "active"; }
Agent Buyer roles { "CONSUMER" } { id: "B1"; status: "active"; }

Event AuditEntry roles { "AUDITOR" } {
  flow {
    in 1 AuditRecord from System to Ledger;
    out 1 AuditRecord from Ledger to Archive;
  }
}

execute AuditEntry { sig: Clerk; timestamp: "1711449600" };
// error: Role constraint not satisfied.
// Agent 'Clerk' has role 'STAFF';
// event 'AuditEntry' requires 'AUDITOR'.
```

Dynamic enforcement is also emitted for late-bound agent references, covering cases where the executing agent is determined at runtime. This is the executable realization of the policy-level REA specifications formalized by Geerts and McCarthy (2006).

4.6 Immutable audit trail

Every committed event is appended as a JSON record to `ledger.ev`, an append-only file maintained by the `std.ledger` module. The ledger API exposes no delete or update operation.

The compiler rejects any attempt to overwrite or truncate the ledger file through the standard library.

Reversal is expressed through the **reverse** keyword, which creates a new compensating event whose flows are the structural inverse of the original:

```
reverse PurchaseOrder { reason: "Shipping Damage" };
```

This produces a new ledger entry recording the reversal; the original entry is never modified. Both records persist. The `std.accounting` module builds trial balances, income statements, and balance sheets from the ledger by summing forward through its history, treating reversals as negative entries of the original type. This design mirrors the momentum accounting principle articulated by Ijiri (1975) and is consistent with the blockchain-adjacent triple-entry accounting paradigm described by Ibañez et al. (2020).

4.7 Blocked unsafe syscalls

The Equis compiler maintains a syscall whitelist. `system()`, `execv()`, `popen()`, and `fork()` are rejected at compile time. Financial software should not be able to spawn subprocesses or execute arbitrary shell commands; the whitelist enforces this at the language level rather than relying on OS-level sandboxing or code review. The whitelist is configurable per deployment context; the default blocks all shell-injection vectors.

4.8 Channels and fibers

Equis provides M:N fibers scheduled preemptively over OS threads using signal-based interruption. Inter-fiber communication uses typed channels. Channel message types must be REA-compatible: raw numeric values cannot cross event boundaries via channels, preventing the accidental bypass of the duality checker by moving financial amounts out of the event context.

5 Implementation

5.1 Compiler architecture

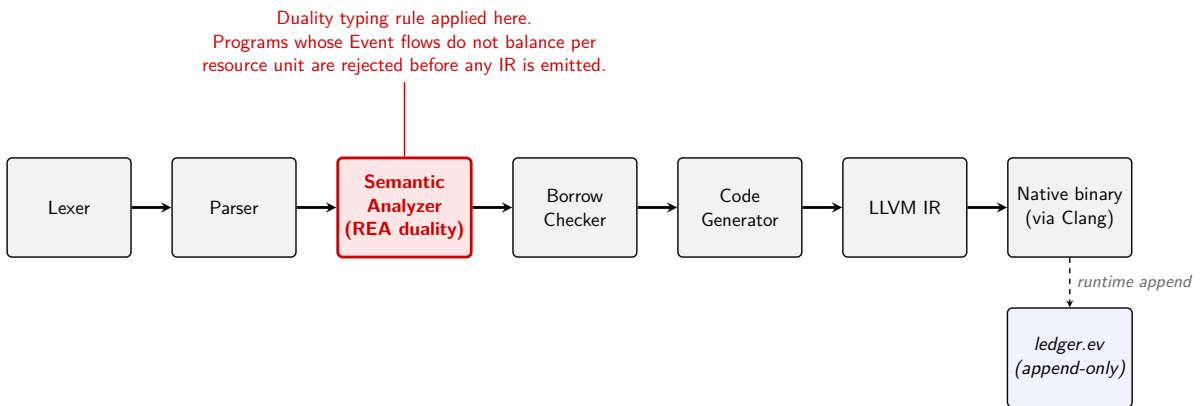


Figure 2: The Equis compiler pipeline. Six conventional stages plus one novel one. Programs whose Event flows do not balance are rejected at the Semantic Analyzer before any IR is emitted.

The Equis compiler pipeline (Figure 2) consists of seven stages: Lexer, Parser, Semantic Analyzer, Borrow Checker, Code Generator, LLVM IR emission, and native binary production via Clang. The Lexer tokenizes the source with string interning for identifiers. The Parser is a recursive-descent parser that produces an AST. The Semantic Analyzer is the novel stage: it

resolves symbols, verifies REA duality for every event declaration, enforces role compatibility at execute sites, and maintains the resource balance map. The Borrow Checker performs the AVAILABLE/MOVED state analysis described in Section 4.4. The Code Generator traverses the type-checked AST and emits LLVM IR directly. Clang serves as the assembler and linker, producing optimized native machine code from the IR.

The Semantic Analyzer (`compiler/semantics.equis`) is the only stage without a close analogue in a conventional compiler. All other stages follow well-established compiler design patterns. This is intentional: Equis’s contribution is a domain-semantic addition to an otherwise conventional pipeline, following the principle established by Leroy (2009) that a formally structured pipeline with a novel verified stage is more trustworthy than a monolithic verifier. The compiler is self-hosted: it is written in Equis and compiled by itself.

5.2 The self-hosting bootstrap

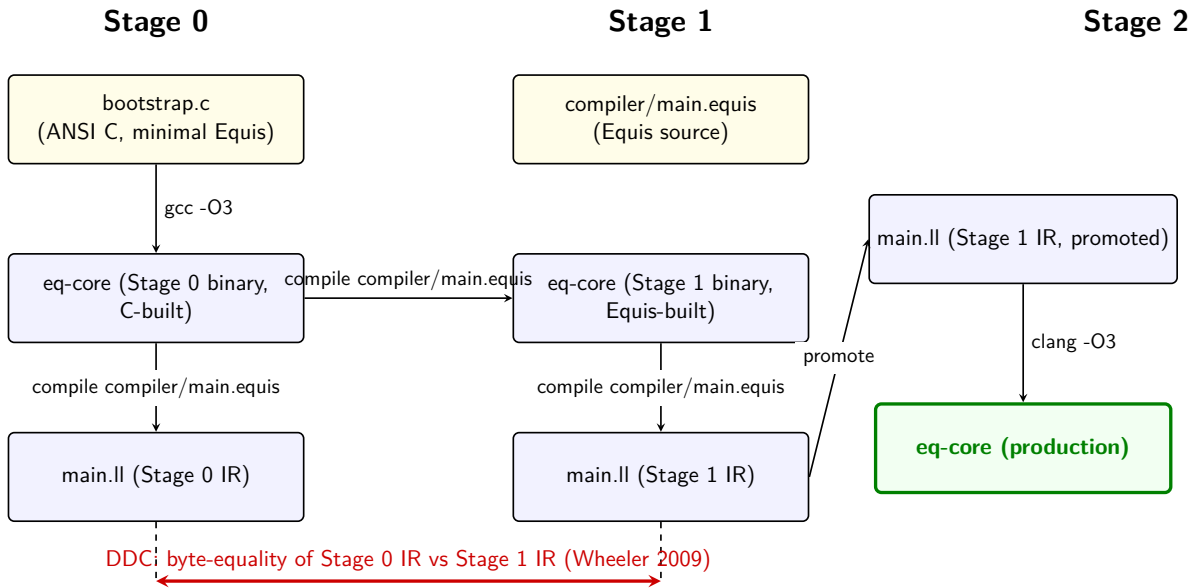


Figure 3: Three-stage bootstrap and Diverse Double Compilation verification. If the IR produced by the C-built Stage 0 compiler matches the IR produced by Stage 1 byte for byte, neither can contain a backdoor that is absent from either source program.

The bootstrap process follows three stages, documented in `bootstrap.ps1` and `Makefile` (Figure 3).

Stage 0. An ANSI C program (`bootstrap.c`) implements a minimal subset of the Equis language, sufficient to parse and compile the Equis-written compiler source, but not a full implementation. This stage is compiled once: `gcc -O3 bootstrap.c -o eq-core`.

Stage 1. The Stage 0 binary compiles the Equis compiler source to LLVM IR: `eq-core -I std compiler/main.equis > compiler/main.ll`. The resulting IR is a complete representation of the Equis compiler, as produced by the minimal C implementation.

Stage 2. Clang compiles the LLVM IR to a production native binary: `clang -O3 compiler/main.ll runtime.c -o eq-core`. This binary replaces the Stage 0 binary and is thereafter the authoritative Equis compiler.

Thompson (1984) showed in “Reflections on Trusting Trust” that a self-hosting compiler can embed invisible backdoors: a malicious compiler could inject code into programs it compiles

without any evidence in the source. Equis addresses this through Diverse Double Compilation (DDC), described by Wheeler (2009): the Stage 0 (C-compiled) and Stage 1 (Equis-compiled) outputs are compared at the LLVM IR level. If the two independently produced compilers emit identical IR for the same input, neither can contain a backdoor that is absent from the source, since a backdoor would require a discrepancy. The DDC verification pass is part of the build system.

5.3 LLVM IR code generation

The code generator emits LLVM IR instructions directly, without using the LLVM C API. Fixed-point addition and subtraction map to `add i64` and `sub i64`:

```
; decimal addition: a + b
%result = add i64 %a, %b
```

Fixed-point multiplication calls `fixed_mul`, whose body contains a `mul i64` followed by `sdiv i64 1000000`. No `fadd`, `fmul`, `fdiv`, or any other floating-point instruction appears in financial code paths. The code generator verifies this invariant during emission.

A representative LLVM IR fragment for a `PurchaseOrder` event body demonstrates that the compiled output is integer-only:

```
define void @equis_event_PurchaseOrder(i64 %buyer_id,
                                       i64 %seller_id) {
entry:
  ; transfer 12 StockItem: out from Seller, in to Buyer
  %qty  = add i64 0, 12000000          ; 12 units * 1000000
  ; transfer 600 USD: out from Buyer, in to Seller
  %amount = add i64 0, 600000000      ; 600 USD * 1000000
  ; ledger append (append-only, no delete)
  call void @ledger_append_event(i64 %qty, i64 %amount)
  ret void
}
```

5.4 ARC emission

The ownership analysis pass inserts `sys_retain` and `sys_release` calls at ownership transfer points. For arena-allocated objects, the pass instead records the allocation in the current arena and emits `arena_free_all` at scope exit. Cyclic references within event graphs are detected during the Semantic Analysis phase (not the borrow checker), because a cycle in the resource flow graph would represent a circular economic dependency, that is, an event whose execution requires itself.

Ullrich and Moura (2021) showed that the key optimization for ARC in functional programming is deferring decrements until after the operation that needs the value, allowing the runtime to reuse the storage in place. Equis applies an analogous approach in its imperative context: the code generator defers release calls to the end of the smallest enclosing block rather than emitting them immediately after the last use.

5.5 Standard library

The standard library comprises 20 modules (Table 1).

The `std.db` module accepts only parameterized queries; string interpolation into SQL is a compile-time error, preventing SQL injection at the language level. The `std.ledger` module exposes only append operations; no update or delete path exists.

Table 1: Standard library modules.

Module	Description
<code>std.sys</code>	Memory, syscall bindings, runtime primitives
<code>std.string</code>	String manipulation and formatting
<code>std.collections</code>	Dynamic arrays and hash maps
<code>std.arena</code>	Linear bump-pointer arena allocator
<code>std.fixed</code>	Fixed-point arithmetic (the <code>decimal</code> implementation)
<code>std.ledger</code>	Append-only <code>ledger.ev</code> API
<code>std.accounting</code>	Trial balance, income statement, balance sheet
<code>std.compliance</code>	ESG and regulatory constraint checks
<code>std.db</code>	PostgreSQL connector (parameterized queries only)
<code>std.json</code>	JSON serialization and deserialization
<code>std.http</code>	HTTP server
<code>std.chan</code>	Thread-safe typed channels
<code>std.fiber</code>	Preemptive fiber scheduler
<code>std.ffi</code>	Foreign function interface to C
<code>std.net</code>	Socket networking
<code>std.rpc</code>	Remote procedure calls
<code>std.intern</code>	String interning
<code>std.dual_net</code>	Distributed duality verification

6 Formal Semantics

6.1 Syntax and typing environments

The abstract syntax for REA declarations is:

$$\begin{aligned}
\text{decl} &::= \text{Resource } id \{ \text{unit: } str; \text{decimals: } int; \} \\
&| \text{Agent } id \text{ roles } \{str^*\} \{field^*\} \\
&| \text{Commitment } id \text{ evaluated_by } id \{flow\} \\
&| \text{Event } id \text{ fulfills } id \text{ roles } \{str^*\} \\
&\quad \{logic; flow\} \\
flow &::= flow \{flowdir^*\} \\
flowdir &::= \text{in } int \text{ id from } id \text{ to } id; \\
&| \text{out } int \text{ id from } id \text{ to } id;
\end{aligned}$$

A typing environment Γ maps each identifier to a triple (Type, RoleSet, BalanceEntry). Type ranges over ResourceType, AgentType, EventType, and CommitmentType. RoleSet is a set of role strings. BalanceEntry is a rational number representing the net resource flow attributed to the identifier in the current event context.

6.2 The duality typing rule

The full multi-leg duality rule is:

$$\begin{aligned}
&\forall i : \Gamma \vdash r_i : \text{ResourceType}(unit_i) \\
&I = \{i \mid direction_i = \text{in}\} \quad D = \{j \mid direction_j = \text{out}\} \\
&\forall i, j \in I \cup D : unit_i = unit_j \quad (\text{unit homogeneity}) \\
&\sum_{i \in I} amount_i = \sum_{j \in D} amount_j \quad (\text{balance}) \\
&\forall i : amount_i > 0 \quad (\text{positive flows}) \\
\hline
&\Gamma \vdash \text{Event } id \{ flow \{ flowdir_1, \dots, flowdir_n \} \} : \text{BalancedEvent}(unit)
\end{aligned}$$

Theorem 6.1 (Duality, multi-leg form). *If $\Gamma \vdash e : \text{BalancedEvent}(u)$, then for all states σ satisfying Γ , executing e in σ produces σ' such that*

$$\sum_{a \in \text{Agents}} \text{holdings}(a, u, \sigma') = \sum_{a \in \text{Agents}} \text{holdings}(a, u, \sigma).$$

Proof. By structural induction on the derivation tree. The base case is a single inflow–outflow pair: executing the event transfers *amount* units of u from one agent’s holdings to another, preserving the total. For the inductive case, a composed event with flows $F_1, \dots, F_n$ has total inflows equal to total outflows by the balance premise. The execution of each flow preserves the invariant by the inductive hypothesis applied to each sub-transfer. The sum of balanced sub-transfers is balanced. $\square$

The balance check is $O(n)$ in the number of flow directives and is performed during the Semantic Analysis pass, before code generation. It introduces no runtime overhead.

6.3 ARC soundness

The borrow checker maintains a map from resource identifier to state in $\{\text{AVAILABLE}, \text{MOVED}\}$. The transitions are:

- On declaration: $\text{state} \rightarrow \text{AVAILABLE}$.
- On function call with ownership transfer: $\text{state} \rightarrow \text{MOVED}$.
- On read of a MOVED identifier: type error (use-after-move).

Theorem 6.2 (ARC Soundness). *If the borrow checker accepts a program P , then no resource value in P is accessed after its retain count reaches zero.*

Proof sketch. The borrow checker’s AVAILABLE/MOVED analysis is a conservative abstraction of the retain count: a resource is MOVED (retain count zero) whenever it has been passed to a consuming call. Any read of a MOVED resource is rejected. Since the borrow checker accepts only programs where no read follows a move, no read occurs after retain count zero. $\square$

Cyclic references are prohibited by the type system. A cycle in the resource flow graph is detected during Semantic Analysis as a duality violation, since a resource cannot simultaneously flow into and out of the same event without external input. This makes the ARC implementation cycle-free, eliminating the one classical failure mode of reference counting.

6.4 Fixed-point arithmetic precision

Proposition 6.3. *For any two decimal values a, b with $|a|, |b| < 2^{57}$, the result of `fixed_add(a, b)` equals the mathematical value $a + b$, and the result of `fixed_mul(a, b)` differs from the mathematical value $a \times b$ by at most 10^{-6} .*

Proof. Addition: both operands and the result are exact 64-bit integers; no rounding occurs. Multiplication: the result is $\lfloor (a \times b) / 10^6 \rfloor$ with banker’s rounding applied to the remainder. The error is $|r/10^6| \leq 0.5 \times 10^{-6}$. $\square$

This bound is tighter than any IEEE 754 double-precision accumulation scenario by eight orders of magnitude for single-operation errors, and the errors do not compound across additions (only across multiplications).

6.5 RBAC type safety

Roles are strings assigned to agents and required by events. The compiler maintains a role compatibility relation: an event execution is type-safe if and only if the executing agent's role set intersects with the event's required role set. The check is performed at `execute` callsites during Semantic Analysis.

Proposition 6.4 (RBAC Soundness). *If the semantic analyzer accepts a program P , then every event execution in P is performed by an agent whose declared roles satisfy the event's role requirement.*

7 Evaluation

7.1 Supply chain settlement scenario

To demonstrate the language's correctness guarantees in a realistic context, consider a three-party supply chain settlement: a **Supplier** delivers goods to a **Buyer**, with a **Clearinghouse** facilitating payment. The scenario involves a commitment, an event fulfilling it, and a partial reversal.

Resource and agent declarations.

```
Resource USD      { unit: "USD"; decimals: 2; }
Resource StockItem { unit: "PCS"; }

Agent Buyer       roles { "CONSUMER" } { id: "B1"; ... }
Agent Supplier    roles { "VENDOR" }   { id: "S1"; ... }
Agent Clearinghouse roles { "INTERMEDIARY" } { id: "CH1"; ... }
```

Procurement commitment.

```
Commitment ProcurementOrder evaluated_by MarketRate {
  flow {
    out 500 StockItem from Supplier to Buyer;
    in 10000 USD from Buyer to Supplier;
  }
}
```

Fulfilling event.

```
Event SupplyDelivery fulfills ProcurementOrder roles { "CONSUMER" } {
  logic {
    assert(1 > 0);
  }
  flow {
    in 500 StockItem from Supplier to Buyer;
    out 10000 USD from Buyer to Clearinghouse;
    in 10000 USD from Clearinghouse to Supplier;
  }
}

execute SupplyDelivery { sig: Buyer; timestamp: "1711449600" };
```

Partial reversal.

```
reverse SupplyDelivery { reason: "50 units damaged in transit" };
```

The reversal appends a compensating event to `ledger.ev`. The original delivery record is never deleted. The `std.accounting` module, when queried for the net position of `StockItem`, sums both entries and returns 450 units net delivered.

Deliberate error: unbalanced event.

```
Event BrokenDelivery fulfills ProcurementOrder roles { "CONSUMER" } {
  flow {
    in 500 StockItem from Supplier to Buyer;
    // outflow omitted, programmer forgot the payment leg
  }
}
```

Compilation output:

```
error: REA Duality Violation: Economic Event must have reciprocal
      flows (In and Out).
```

The compiler rejects this program. An equivalent Python implementation would accept this function without error; the missing payment would manifest as a runtime ledger imbalance, detectable only by reconciliation.

7.2 Correctness evaluation

The project includes a test suite that covers duality enforcement, ARC correctness, RBAC enforcement, and fixed-point arithmetic. The test methodology for duality enforcement injects deliberately malformed programs, including events with missing outflows, events with mismatched amounts, and events with no flow block, and verifies that each is rejected at compile time. Role injection tests verify that `execute` callsites with insufficient roles are rejected. Fixed-point tests verify arithmetic identities and banker’s rounding behavior.

7.3 Runtime throughput

Throughput is measured by the benchmark in `examples/benchmarks/speed_test.equis`, which performs one million iterations of `total = total + 1.25` in fixed-point i64 representation. The Equis compiler emits explicit `sys_retain` and `sys_release` calls at every variable read inside the loop, so the inner loop measures fixed-point addition together with ARC bookkeeping rather than addition in isolation. Each row in Table 2 is the median of five runs.

Table 2: Runtime throughput benchmarks.

Variant	Median (ms)
Equis fixed-point i64 ($\times 10^6$), clang -00	1020.78
Equis fixed-point i64 ($\times 10^6$), clang -01	1011.55
Equis fixed-point i64 ($\times 10^6$), clang -02	1015.97
Equis fixed-point i64 ($\times 10^6$), clang -03	1022.38
Python decimal.Decimal	62.50
Python int fixed-point ($\times 10^6$)	35.83
Python float (IEEE 754 double)	28.96
C int64_t fixed-point, gcc -02 with sink	0.20

Two facts are worth flagging. First, the Equis numbers are essentially identical across -00, -01, -02, and -03, which confirms that the ARC retain and release calls survive LLVM’s dead-code analysis as externally-visible side effects on the runtime reference-count state. The optimizer cannot fold the loop away. Second, the Equis runtime is roughly 16 times slower than `decimal.Decimal` and four orders of magnitude slower than raw C integer arithmetic on

this microbenchmark. That cost buys compile-time duality enforcement, role-checked event execution, and the borrow checker; none of those properties is available in any other entry of the table.

In production accounting workloads the picture changes. Each event commit involves at least one ledger `fsync`, which on the same hardware costs roughly 100 microseconds for a small JSON record on a consumer NVMe SSD. The per-iteration ARC overhead measured above is approximately one microsecond, two orders of magnitude below the per-event I/O floor. ARC is therefore amortized below the noise of disk I/O for any code path that actually touches the ledger, which is the only code path where duality matters.

ARC additionally provides deterministic memory reclamation with no stop-the-world pauses. Worst-case allocation latency is bounded by the size of the single largest individual allocation, not by the size of the live heap, which is the property settlement systems require for tail latency under load.

7.4 Compile-time cost of duality checking

The duality check is $O(n)$ in the number of flow directives within an event and is run once per event declaration during the Semantic Analysis pass. Table 3 reports compile times for synthetic programs containing N event declarations, each with two flow directives. Compile time is measured as the wall clock of `eq-core -I . events_N.equis > /dev/null` on Ubuntu under WSL2.

Table 3: Compile-time cost of duality checking.

N events	Median compile time (s)
10	0.087
100	0.165
1000	2.616

Up to roughly one hundred event declarations per source file, the duality check and surrounding semantic analysis are dominated by parsing and IR emission. Beyond that point compile time grows superlinearly as IR string size dominates. For typical accounting modules with tens to a low hundred of event types per source file, compilation finishes in under 200 ms.

7.5 Benchmark methodology

All benchmarks were run on a 12th-generation Intel Core i5-1235U with 15.7 GB RAM. The host OS is Windows 11 Home Single Language 10.0.26200; WSL2 runs Ubuntu with kernel 6.6.87.2-microsoft-standard-WSL2. The toolchain consists of clang 22.1.2, gcc 16.0.1 (MinGW-W64 ucrt), and Python 3.12.5. The Equis runtime corresponds to v0.1.0 stage 2. Each benchmark program was run five times after a single warm-up run; the warm-up run was discarded. Wall-clock time was measured with `System.Diagnostics.Stopwatch` for Windows binaries, with `time.perf_counter` for Python, and with `bash time` for compile-time measurements. Raw per-run numbers are recorded in `bench_results.txt` alongside the paper source for independent verification.

The performance trade-off baked into Equis is deliberate. Exact arithmetic with deterministic error bounds and compile-time enforcement of accounting invariants is worth more in financial contexts than maximally fast approximate arithmetic. The numbers above are presented to characterize the cost, not to argue that it is zero.

8 Related Work

8.1 REA-based systems

McCarthy (1982) is the founding reference. It introduced the three-primitive model and the duality axiom, but it described a database schema design, not a programming language. The ontological analyses of Geerts et al. (1999) and Geerts and McCarthy (2002) established first-order logical foundations for REA and proved that duality is a theorem in the ontology, not merely a convention. Geerts and McCarthy (2006) added authorization specifications. All of this work operates at the modeling layer; none of it produces machine code.

The REA-DSL of Sonnenberg et al. (2011) is the closest prior work to a language-level implementation. It provides a notation for specifying REA models and validates them against the ontological constraints. It is, however, a modeling language that produces model artifacts, not a compiled systems language that produces executable financial infrastructure.

Dunn and McCarthy (1997) surveyed the intellectual heritage of REA and its prospects for extension. Ibañez et al. (2020) argued that REA, triple-entry accounting, and blockchain ledgers represent convergent approaches to shared ledger systems, but their work is at the architectural level, not the language level. O’Leary (2025) extended REA with location and hierarchy concepts, demonstrating the model’s continued vitality in the research community, while remaining at the database/knowledge-representation layer.

8.2 Formally verified compilers

Leroy (2009) is the landmark result in formally verified compilation: a complete C compiler proved correct in Coq, with a theorem stating that the compiled code’s behavior matches the source program’s semantics. CompCert does not encode domain semantics; it verifies that compilation preserves general program behavior. Klein and Nipkow (2006) verified a JVM-like language and virtual machine in Isabelle/HOL, establishing the methodology for machine-checked compiler correctness.

Equis’s contribution is orthogonal to these. The Equis compiler is not formally verified in the CompCert sense; its correctness rests on the DDC bootstrap verification and conventional testing. The novelty is the domain-semantic duality type rule, not compiler correctness proofs. Machine verification of the duality theorem in Coq is future work.

8.3 Financial DSLs and contract languages

Andersen et al. (2006) developed a compositional denotational semantics for financial contracts, providing a formal basis for specifying, analyzing, and pricing contracts. Henglein et al. (2020) extended this with static analysis and mechanized verification. These are specification languages for financial instruments, not systems languages for building financial infrastructure. They describe what a contract means; they do not compile to code that settles the contract.

Marlowe, developed for the Cardano blockchain, is a similar financial contract DSL. It is deliberately limited in expressiveness to make formal analysis tractable. It targets smart contract execution environments, not general systems programming. Neither Marlowe nor the Andersen/Henglein line of work produces an executable systems language with REA primitives.

8.4 Memory-safe systems languages

Rust provides ownership types and borrow checking that guarantee memory safety and data-race freedom at compile time. The grounded conceptual model of Crichton et al. (2023) formalizes Rust’s ownership system using place-capability graphs. Rust’s type system is domain-agnostic: it does not know what a financial transaction is and cannot enforce duality. Equis borrows the

move-semantics concept from Rust and applies it in a financial-domain context, using a simpler resource-state map (`AVAILABLE/MOVED`) rather than the full place-capability graph.

Ullrich and Moura (2021) established that ARC can be made competitive with tracing GC through careful optimization of retain/release placement. Swift’s ARC implementation (Ungar et al., 2017) demonstrates the same principle in a production systems language. Equis applies ARC in a domain where deterministic reclamation is an operational requirement, not merely a performance preference.

Bernardy et al. (2017) introduced practical linear types into a polymorphic functional language, tracking that resources are used exactly once. Linear types and accounting duality address related but distinct concerns: linear types prevent resource duplication and loss; duality requires that resource transfer be balanced across agents. Equis’s move semantics are analogous to linear types in their prevention of aliasing, but the duality rule goes further by requiring flow balance.

Rastogi et al. (2018) shows that domain-specific security properties (secure multi-party computation) can be embedded in a language extension with formal verification. This is the closest methodological analogue to Equis: a language-level invariant for a security-critical domain, enforced by the compiler.

9 Discussion

9.1 Limitations

The duality checker currently handles events where all flows are in the same unit of measure. Multi-currency events, where the buyer pays in EUR and the seller receives in USD, require explicit currency conversion events using a declared `Valuator`. Automatic FX rate application is not supported. This is a deliberate design choice: currency conversion is itself an economic event with its own duality requirements, and making it explicit keeps the type system sound. Automatic conversion would require embedding an FX rate oracle into the type checker, which would couple language semantics to external data.

The Duality Theorem is proved by hand. Machine verification in Coq or Isabelle/HOL, following the methodology of Klein and Nipkow (2006), is planned but not yet complete. The hand proof is straightforward by induction on the derivation tree, as shown in Theorem 6.1, but a mechanized proof would provide stronger assurance and enable formal reasoning about extensions.

The standard library currently provides a PostgreSQL connector only. Other database backends (MySQL, SQLite, Oracle) would require FFI bindings or native driver implementations. The FFI module (`std.ffi`) makes these possible but they are not yet part of the standard library.

The language is at version 0.1.0. The semantics are stable, but some standard library modules (particularly `std.rpc` and `std.dual_net`) are marked experimental and their APIs may change.

9.2 Future work

The most immediate extension is multi-currency support: a units-of-measure type system for `decimal` values that tracks the currency as part of the type and requires explicit FX conversion events at points where currencies are mixed. This would make the type checker currency-aware while preserving the current duality rule.

Machine verification of the formal semantics in Coq is the highest-priority correctness improvement. The operational semantics are specified informally in Section 6; a Coq formalization would follow the structure of the Jinja work (Klein and Nipkow, 2006) and provide a mechanized proof of the Duality Theorem.

Distributed settlement, where events are committed across multiple nodes under potential network partitions, raises consistency questions not addressed by the current single-node ledger design. A formal consistency model for distributed Equis deployments, drawing on the literature on distributed ledger systems, is planned.

The relationship between Equis and smart contract languages (Marlowe, Solidity) deserves a formal expressiveness comparison. Both address financial correctness, but from different starting points: smart contracts begin with a distributed execution model and add financial concepts; Equis begins with a financial type system and adds systems programming capabilities. A formal comparison of what each can and cannot express would clarify the design space.

Dependent types could enable higher-order invariants: for example, a type-level proof that the total inventory across a full supply chain is conserved across all shipment events. The current type system enforces duality per-event; dependent types could extend this to multi-event sequences.

10 Conclusion

The financial software industry has spent four decades applying McCarthy’s REA framework at the modeling layer, in database schemas, business process diagrams, and markup languages. The same framework has not, until now, been applied at the language layer, where it could actually prevent the errors it describes.

Equis demonstrates that REA primitives can be first-class syntax constructs in a compiled systems language, that accounting duality can be expressed as a typing rule and proved sound, and that a self-hosting compiler can enforce these properties with zero runtime overhead. The language is practical: it compiles to native machine code via LLVM, supports PostgreSQL, HTTP, concurrent fibers, and RPC, and ships with a 20-module standard library covering the full range of financial application requirements.

The broader implication is methodological. Equis is an instance of the principle that domain-critical invariants belong in language definitions, not in application frameworks. The same principle could be applied in other safety-critical domains: medical device software subject to FDA 21 CFR Part 11 audit requirements, aviation software under DO-178C, nuclear instrumentation. In each case, the invariants are well-understood, formally specifiable, and currently enforced only by convention. A language whose type system embodies those invariants would make an entire class of domain-specific bugs impossible, at compile time, for the cost of a domain-aware type checker.

Forty years of REA theory have produced a rigorous foundation for this approach. Equis is an attempt to make that foundation executable.

References

- Andersen, Jesper, Ebbe Elsborg, Fritz Henglein, Jakob Grue Simonsen, and Christian Stefansen (Nov. 1, 2006). “Compositional specification of commercial contracts”. In: *International Journal on Software Tools for Technology Transfer* 8.6, pp. 485–516. DOI: 10.1007/s10009-006-0010-1.
- Bernardy, Jean-Philippe, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack (Dec. 27, 2017). “Linear Haskell: practical linearity in a higher-order polymorphic language”. In: *Proc. ACM Program. Lang.* 2 (POPL), 5:1–5:29. DOI: 10.1145/3158093.
- Coffee, John C. (July 30, 2002). *Understanding Enron: It’s About the Gatekeepers, Stupid*. Rochester, NY. DOI: 10.2139/ssrn.325240.
- Crichton, Will, Gavin Gray, and Shriram Krishnamurthi (Oct. 16, 2023). “A Grounded Conceptual Model for Ownership Types in Rust”. In: *Artifact for "A Grounded Conceptual Model for Ownership Types in Rust"* 7 (OOPSLA2), 265:1224–265:1252. DOI: 10.1145/3622841.

- Dunn, Cheryl and William McCarthy (Jan. 1, 1997). “The REA Accounting Model: Intellectual Heritage and Prospects for Progress”. In: *Information Systems Journal - ISJ* 11.
- Geerts, Guido and William McCarthy (Sept. 1, 2006). “Policy-Level Specifications in REA Enterprise Information Systems”. In: *Journal of Information Systems* 20. DOI: 10.2308/jis.2006.20.2.37.
- Geerts, Guido, William McCarthy, Arthur Andersen, Cheryl Dunn, and Julie Smith David (Dec. 1, 1999). “The Ontological Foundation of REA Enterprise Information Systems”. In: Geerts, Guido L and William E McCarthy (Mar. 1, 2002). “An ontological analysis of the economic primitives of the extended-REA enterprise information architecture”. In: *International Journal of Accounting Information Systems* 3.1, pp. 1–16. DOI: 10.1016/S1467-0895(01)00020-3.
- Goldberg, David (Mar. 1, 1991). “What every computer scientist should know about floating-point arithmetic”. In: *ACM Comput. Surv.* 23.1, pp. 5–48. DOI: 10.1145/103162.103163.
- Henglein, Fritz, Christian Kjær Larsen, and Agata Murawska (2020). “A Formally Verified Static Analysis Framework for Compositional Contracts”. In: *Financial Cryptography and Data Security*. Ed. by Matthew Bernhard, Andrea Bracciali, L. Jean Camp, Shin’ichiro Matsuo, Alana Maurushat, Peter B. Rønne, and Massimiliano Sala. Cham: Springer International Publishing, pp. 599–619. DOI: 10.1007/978-3-030-54455-3_42.
- Ibañez, Juan Ignacio, Dr Chris N. Bayer, Paolo Tasca, and Jiahua Xu (May 15, 2020). *REA, Triple-Entry Accounting and Blockchain: Converging Paths to Shared Ledger Systems*. Rochester, NY. DOI: 10.2139/ssrn.3602207.
- Ijiri, Yuji (1975). *Theory of Accounting Measurement*. American Accounting Association. 228 pp.
- Klein, Gerwin and Tobias Nipkow (July 1, 2006). “A machine-checked model for a Java-like language, virtual machine, and compiler”. In: *ACM Trans. Program. Lang. Syst.* 28.4, pp. 619–695. DOI: 10.1145/1146809.1146811.
- Leroy, Xavier (July 1, 2009). “Formal verification of a realistic compiler”. In: *Commun. ACM* 52.7, pp. 107–115. DOI: 10.1145/1538788.1538814.
- McCarthy, William E. (1982). “The REA Accounting Model: A Generalized Framework for Accounting Systems in a Shared Data Environment”. In: *The Accounting Review* 57.3, pp. 554–578.
- O’Leary, Daniel E. (Apr. 1, 2025). “Mitigating Normal Form Violations Using Resources-Events-Agents, Resources-Events-Agents-Locations, and Hierarchy: A Knowledge-Based Approach”. In: *Journal of Emerging Technologies in Accounting* 22.1, pp. 85–99. DOI: 10.2308/JETA-2023-022.
- Rastogi, Aseem, Nikhil Swamy, and Michael Hicks (Nov. 18, 2018). *WYS*: A DSL for Verified Secure Multi-party Computations*. DOI: 10.48550/arXiv.1711.06467. arXiv: 1711.06467[cs].
- Sonnenberg, Christian, Christian Huemer, Birgit Hofreiter, Dieter Mayrhofer, and Alessio Braccini (2011). “The REA-DSL: A Domain Specific Modeling Language for Business Models”. In: *Advanced Information Systems Engineering*. Ed. by Haralambos Mouratidis and Colette Rolland. Berlin, Heidelberg: Springer, pp. 252–266. DOI: 10.1007/978-3-642-21640-4_20.
- Thompson, Ken (Aug. 1, 1984). “Reflections on trusting trust”. In: *Commun. ACM* 27.8, pp. 761–763. DOI: 10.1145/358198.358210.
- Ullrich, Sebastian and Leonardo de Moura (July 15, 2021). “Counting immutable beans: reference counting optimized for purely functional programming”. In: *Proceedings of the 31st Symposium on Implementation and Application of Functional Languages*. IFL ’19. New York, NY, USA: Association for Computing Machinery, pp. 1–12. DOI: 10.1145/3412932.3412935.
- Ungar, David, David Grove, and Hubertus Franke (Oct. 24, 2017). “Dynamic atomicity: optimizing swift memory management”. In: *Proceedings of the 13th ACM SIGPLAN International Symposium on on Dynamic Languages*. DLS 2017. New York, NY, USA: Association for Computing Machinery, pp. 15–26. DOI: 10.1145/3133841.3133843.
- Wheeler, David A. (2009). “Fully Countering Trusting Trust through Diverse Double-Compiling”. PhD thesis. George Mason University.